

**UNIVERSIDADE DE SÃO PAULO
ESCOLA DE ENGENHARIA DE SÃO CARLOS**

Caio Wingeter de Castilho

**Otimização de Baixo Nível em Vision Transformers:
Desenvolvimento de Kernels de Inferência Customizados
para Eficiência de SRAM**

São Carlos

2026

Caio Wingeter de Castilho

**Otimização de Baixo Nível em Vision Transformers:
Desenvolvimento de Kernels de Inferência Customizados
para Eficiência de SRAM**

Monografia apresentada ao Curso de Engenharia Aeronáutica, da Escola de Engenharia de São Carlos da Universidade de São Paulo, como parte dos requisitos para obtenção do título de Engenheiro Aeronáutico.

Orientador: Prof. Dr. Glauco Caurin

VERSÃO CORRIGIDA

**São Carlos
2026**

É possível elaborar a ficha catalográfica em LaTeX ou incluir a fornecida pela Biblioteca. Para tanto observe a programação contida nos arquivos USPSC-modelo.tex e fichacatalografica.tex e/ou gere o arquivo fichacatalografica.pdf.

A biblioteca da sua Unidade lhe fornecerá um arquivo PDF com a ficha catalográfica definitiva, que deverá ser salvo como fichacatalografica.pdf no diretório do seu projeto.

Folha de aprovação em conformidade
com o padrão definido
pela Unidade.

No presente modelo consta como
folhadeaprovacao.pdf

*Este trabalho é dedicado à todos que me apoiaram
e me deram o suporte durante esse percurso.*

AGRADECIMENTOS

Dedico este trabalho a todos que estiveram ao meu lado ao longo dessa caminhada, oferecendo apoio, força e incentivo para que eu pudesse superar cada desafio encontrado no percurso.

Agradeço, de forma especial, à minha companheira Luana, por todo o carinho, paciência e apoio incondicional ao longo dessa jornada. Aos meus pais, André e Carolina, que, apesar de todas as dificuldades, nunca deixaram de acreditar em mim e sempre se esforçaram ao máximo para que eu pudesse alcançar meus sonhos. Ao meu irmão Breno, que esteve ao meu lado em todos os momentos, não apenas como irmão, mas como meu melhor amigo.

*“Líderes são mestres em encontrar o sentido e aprendizado
em todo tipo de experiência de vida.”*
(Jucá, 2014, p. 94)

RESUMO

CASTILHO, C. W. **Otimização de Baixo Nível em Vision Transformers: Desenvolvimento de Kernels de Inferência Customizados para Eficiência de SRAM.** 2026. 66 p. Monografia (Trabalho de Conclusão de Curso) - Escola de Engenharia de São Carlos, Universidade de São Paulo, São Carlos, 2026.

Este trabalho aborda a otimização de baixo nível de kernels de inferência aplicados a modelos de visão computacional, com foco na arquitetura Vision Transformer (ViT), motivada pelas restrições de latência e energia da percepção embarcada em Veículos Aéreos Não Tripulados (VANTs). O objetivo principal é reduzir o gargalo de transferência de dados entre a memória global e os Streaming Multiprocessors (SM) da GPU, fenômeno associado à “Memory Wall”, particularmente severo no hardware de borda que equipa esses veículos. Para isso, desenvolve-se um kernel de atenção fundido em OpenAI Triton, explorando tiling, reuso de dados em registradores e SRAM, e cálculo de softmax online para evitar a materialização completa da matriz intermediária de atenção. A validação compara a implementação customizada com baselines em PyTorch por meio de latência, throughput, erro numérico, tráfego de memória e análise pelo Modelo Roofline. Os resultados indicam que a fusão de operadores reduz substancialmente o tráfego em memória global e acelera a execução em relação à implementação explícita, embora permaneça abaixo do backend otimizado do PyTorch. A partir das intensidades aritméticas medidas, discutem-se as implicações para plataformas embarcadas típicas de VANTs, nas quais a menor banda de memória torna a fusão de operadores proporcionalmente mais vantajosa. O trabalho contribui com uma implementação experimental e um protocolo de análise para investigar gargalos de desempenho em kernels de atenção aplicados a Vision Transformers.

Palavras-chave: Visão Computacional. Vision Transformers. Otimização de Kernels. OpenAI Triton. Engenharia de Performance. Computação Embarcada. VANTs.

ABSTRACT

CASTILHO, C. W. **Hardware-Aware Optimization of Vision Transformers: Implementing High-Performance Inference Kernels for SRAM Efficiency.** 2026. 66 p. Monograph (Conclusion Course Paper) - Escola de Engenharia de São Carlos, Universidade de São Paulo, São Carlos, 2026.

This work addresses the low-level optimization of inference kernels applied to computer vision models, focusing on the Vision Transformer (ViT) architecture, motivated by the latency and energy constraints of onboard perception in Unmanned Aerial Vehicles (UAVs). The primary objective is to reduce the data transfer bottleneck between global memory and the GPU's Streaming Multiprocessors (SM), a phenomenon associated with the "Memory Wall," which is particularly severe on the edge hardware carried by such vehicles. To this end, a fused attention kernel is developed using OpenAI Triton, exploring tiling, data reuse in registers and SRAM, and online softmax computation to avoid fully materializing the intermediate attention matrix. The validation compares the custom implementation against PyTorch baselines using latency, throughput, numerical error, memory traffic, and Roofline analysis. The results indicate that operator fusion substantially reduces global memory traffic and accelerates execution relative to the explicit implementation, although it remains below PyTorch's optimized backend. Based on the measured arithmetic intensities, the implications for UAV-class embedded platforms are discussed, showing that the lower memory bandwidth of edge devices makes operator fusion proportionally more advantageous. This work contributes an experimental implementation and an analysis protocol for investigating performance bottlenecks in attention kernels applied to Vision Transformers.

Keywords: Computer Vision. Vision Transformers. Kernel Optimization. OpenAI Triton. Performance Engineering. Edge Computing. UAVs.

LISTA DE FIGURAS

Figura 1 – Arquitetura do Vision Transformer (ViT)	31
Figura 2 – Mecanismos de atenção do Transformer	32
Figura 3 – Hierarquia de memória com largura de banda e capacidade	34
Figura 4 – Esquema do FlashAttention e comparação de tempo de execução . . .	37
Figura 5 – Comparação de latência entre as implementações de atenção	44
Figura 6 – Speedup relativo à implementação PyTorch eager	45
Figura 7 – Throughput efetivo das implementações de atenção	45
Figura 8 – Roofline empírico das implementações de atenção	47

LISTA DE TABELAS

Tabela 1 – Resultados comparativos de latência por comprimento de sequência (mediana de cinco execuções independentes)	43
---	----

LISTA DE QUADROS

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
BF16	Brain Floating Point de 16 bits
CUDA	Compute Unified Device Architecture
DRAM	Dynamic Random Access Memory
EESC	Escola de Engenharia de São Carlos
FLOP	Floating Point Operation
FP16	Ponto flutuante de 16 bits (meia precisão)
FP32	Ponto flutuante de 32 bits (precisão simples)
GPU	Graphics Processing Unit
HBM	High Bandwidth Memory
HPC	High Performance Computing
JIT	Just-In-Time (compilação)
ML	Machine Learning
PDF	Portable Document Format
SDPA	Scaled Dot-Product Attention (função do PyTorch)
SM	Streaming Multiprocessors
SRAM	Static Random Access Memory
SWaP	Size, Weight and Power
TCC	Trabalho de Conclusão de Curso
USP	Universidade de São Paulo
VANT	Veículo Aéreo Não Tripulado
ViT	Vision Transformer

LISTA DE SÍMBOLOS

Q	Matriz <i>Query</i>
K	Matriz <i>Key</i>
V	Matriz <i>Value</i>
K^T	Transposta da matriz K
d_k	Dimensionalidade dos vetores de chave
B	Tamanho do lote (<i>batch</i>)
H	Número de cabeças de atenção
N	Número de <i>patches</i> (tokens) da entrada
D	Dimensão de cada cabeça de atenção
I	Intensidade aritmética (FLOPs por byte)
S_{ij}	Bloco de <i>scores</i> de atenção
m_i	Máximo acumulado da linha no <i>softmax online</i>
l_i	Soma exponencial acumulada no <i>softmax online</i>
O_i	Acumulador vetorial de saída no <i>softmax online</i>
$O(N^2)$	Complexidade computacional quadrática em função de N

SUMÁRIO

1	INTRODUÇÃO	27
1.1	Contexto e delimitação do tema	27
1.2	Justificativa	28
1.3	Problema de pesquisa	28
1.4	Objetivos	29
1.5	Organização do trabalho	29
2	FUNDAMENTAÇÃO TEÓRICA	31
2.1	Vision Transformers e a Mecânica de Atenção	31
2.1.1	Por que Autoatenção?	32
2.2	Complexidade Computacional e Movimentação de Dados na Atenção	33
2.3	Arquitetura de GPU e a “Parede de Memória” (Memory Wall)	34
2.3.1	Modelo de execução em GPUs	35
2.4	Modelo Roofline para Análise de Desempenho	35
2.5	OpenAI Triton	36
2.6	Fusão de Operadores e Flash Attention	36
3	METODOLOGIA	39
3.1	Ambiente Experimental	39
3.2	Design do Kernel Customizado	39
3.3	Implementação do Softmax Online	40
3.4	Protocolo de Avaliação	41
4	RESULTADOS E DISCUSSÕES	43
4.1	Análise Comparativa de Latência	43
4.2	Profiling de Baixo Nível	46
4.3	Modelo Roofline	47
4.4	Implicações para Inferência Embarcada em VANTs	48
5	CONCLUSÃO	51
	REFERÊNCIAS	53

APÊNDICES	55
APÊNDICE A – COMPARAÇÃO ENTRE SOFTMAX PADRÃO E SOFTMAX COM TRITON	57
A.1 Implementação padrão em PyTorch	57
A.2 Implementação otimizada com Triton	58
A.3 Comparação técnica	61
APÊNDICE B – CÓDIGO DO KERNEL DE ATENÇÃO FUNDIDO	63

1 INTRODUÇÃO

O avanço dos modelos de Aprendizado de Máquina (ML) em Visão Computacional tem sido marcado pela adoção de arquiteturas baseadas em Transformers, especialmente os Vision Transformers (ViTs) (Dosovitskiy *et al.*, 2021). Diferente das convoluções, que operam majoritariamente em domínios locais, os ViTs utilizam autoatenção (Self-Attention) para capturar dependências globais em uma imagem (Vaswani *et al.*, 2017). Essa capacidade, porém, impõe alto custo computacional e intensa movimentação de memória, criando desafios para a implantação desses modelos em ambientes de inferência de alto desempenho (Dao *et al.*, 2022).

Esse desafio é particularmente relevante no contexto aeronáutico. Veículos Aéreos Não Tripulados (VANTs) executam, de forma crescente, tarefas de percepção baseadas em aprendizado profundo diretamente a bordo, como detecção de obstáculos, odometria visual, inspeção de estruturas e navegação autônoma em ambientes sem GPS (Loquercio *et al.*, 2021; Kaufmann *et al.*, 2023). Essas cargas de trabalho rodam em computadores embarcados de baixo consumo, como os módulos da família NVIDIA Jetson, sujeitos a restrições de tamanho, peso e potência (SWaP, do inglês *Size, Weight and Power*). Nesse cenário, a latência da percepção precisa caber no período da malha de guiamento e controle do veículo, e cada joule consumido pelo processamento compete diretamente com a autonomia de voo.

1.1 Contexto e delimitação do tema

Na computação de alto desempenho aplicada à Inteligência Artificial, o gargalo nem sempre está na capacidade aritmética das Unidades de Processamento Gráfico (GPUs), mas na eficiência da movimentação de dados. Esse fenômeno, conhecido como Memory Wall, ocorre quando a transferência entre a memória principal e os Streaming Multiprocessors (SM) da GPU não acompanha o ritmo das unidades de execução (Wulf; McKee, 1996). Em plataformas embarcadas, o desequilíbrio é ainda mais acentuado: enquanto uma GPU de desktop recente dispõe de centenas de GB/s de banda de memória dedicada, um módulo como o Jetson Orin Nano oferece cerca de 68 GB/s de banda LPDDR5, compartilhada entre CPU e GPU (NVIDIA Corporation, 2024). Operações limitadas por memória, como a atenção implementada de forma ingênua, são portanto proporcionalmente mais penalizadas no hardware de borda que equipa VANTs.

Frameworks de alto nível, como PyTorch e TensorFlow, oferecem implementações versáteis para operações de redes neurais. Entretanto, quando operações sequenciais são executadas como kernels separados, tensores intermediários podem ser escritos e lidos na memória global, aumentando a latência. Esta pesquisa delimita-se à otimização de baixo

nível da operação de atenção em Vision Transformers, utilizando OpenAI Triton para explorar explicitamente a hierarquia de memória do hardware (Tillet; Kung; Cox, 2019). O desenvolvimento e a avaliação experimental são realizados em uma GPU de desktop, com a mesma pilha de software (PyTorch, Triton, CUDA) disponível nos módulos embarcados da família Jetson; as implicações dos resultados para inferência embarcada em VANTs são discutidas analiticamente por meio do Modelo Roofline, e a validação em hardware embarcado real é apontada como trabalho futuro.

1.2 Justificativa

A escolha do tema conecta a formação em Engenharia Aeronáutica com uma limitação prática dos sistemas aéreos autônomos: o orçamento de latência e energia da percepção embarcada. Três razões sustentam a abordagem em nível de kernel:

1. **O gargalo dominante em hardware embarcado é a memória.** Como a banda de memória dos módulos embarcados é uma ordem de grandeza menor que a de GPUs de desktop, otimizações que reduzem tráfego de memória, como a fusão de operadores, tendem a produzir ganhos proporcionalmente maiores em plataformas de borda do que em estações de trabalho (Dao *et al.*, 2022; NVIDIA Corporation, 2024);
2. **Otimização de kernel preserva o modelo.** Diferentemente de técnicas como quantização agressiva, poda ou substituição de arquitetura (Mehta; Rastegari, 2022; Cai *et al.*, 2023), a otimização em nível de kernel acelera exatamente a mesma computação, sem alterar pesos nem saídas do modelo além de efeitos de arredondamento. Em aplicações aeronáuticas, nas quais o comportamento do sistema de percepção precisa ser validado, essa propriedade é desejável;
3. **Reduzir tráfego de memória reduz energia.** O acesso à DRAM custa ordens de grandeza mais energia que uma operação aritmética no chip (Horowitz, 2014). Em um VANT, menor consumo por inferência traduz-se diretamente em maior autonomia de voo.

1.3 Problema de pesquisa

A operação de atenção em ViTs possui complexidade quadrática em relação ao número de patches da imagem (Vaswani *et al.*, 2017; Dosovitskiy *et al.*, 2021). Em implementações não fundidas, a materialização de matrizes intermediárias de atenção na memória global aumenta a latência e reduz a eficiência do uso da GPU (Dao *et al.*, 2022). Surge, portanto, a questão: como o desenvolvimento de kernels customizados e a aplicação de técnicas de fusão de operadores podem reduzir a dependência da memória

global e aproximar o desempenho dos modelos de visão dos limites efetivos do hardware, em particular do hardware embarcado de baixo consumo utilizado em VANTs?

1.4 Objetivos

O objetivo central deste trabalho é desenvolver, implementar e validar um kernel de inferência de alto desempenho para a operação de autoatenção em Vision Transformers, avaliando suas implicações para inferência embarcada.

Sendo seus objetivos específicos:

- Desenvolver um kernel fundido (fused kernel) utilizando OpenAI Triton que integre as operações de produto escalar, softmax e ponderação de valores em um único kernel, mantendo os dados intermediários em registradores e SRAM;
- Implementar estratégias de tiling para otimizar o carregamento de blocos de dados nos registradores da GPU;
- Realizar um profiling comparativo entre a implementação proposta e baselines em PyTorch, utilizando métricas de latência, throughput (TFLOP/s) e utilização de banda de memória;
- Analisar a eficiência do kernel desenvolvido através do Modelo Roofline para identificar gargalos operacionais remanescentes;
- Discutir, a partir das intensidades aritméticas medidas, as implicações dos resultados para inferência em plataformas embarcadas típicas de VANTs.

1.5 Organização do trabalho

O restante deste documento está organizado da seguinte forma. O Capítulo 2 apresenta a fundamentação teórica: a arquitetura dos Vision Transformers, o custo computacional e de movimentação de dados da atenção, a hierarquia de memória de GPUs, o Modelo Roofline e as ferramentas de programação de baixo nível. O Capítulo 3 descreve o ambiente experimental, o design do kernel customizado e o protocolo de avaliação. O Capítulo 4 apresenta os resultados comparativos de latência, o profiling de baixo nível, a análise Roofline e a discussão das implicações para inferência embarcada em VANTs. Por fim, o Capítulo 5 retoma os objetivos, sintetiza as contribuições e aponta trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

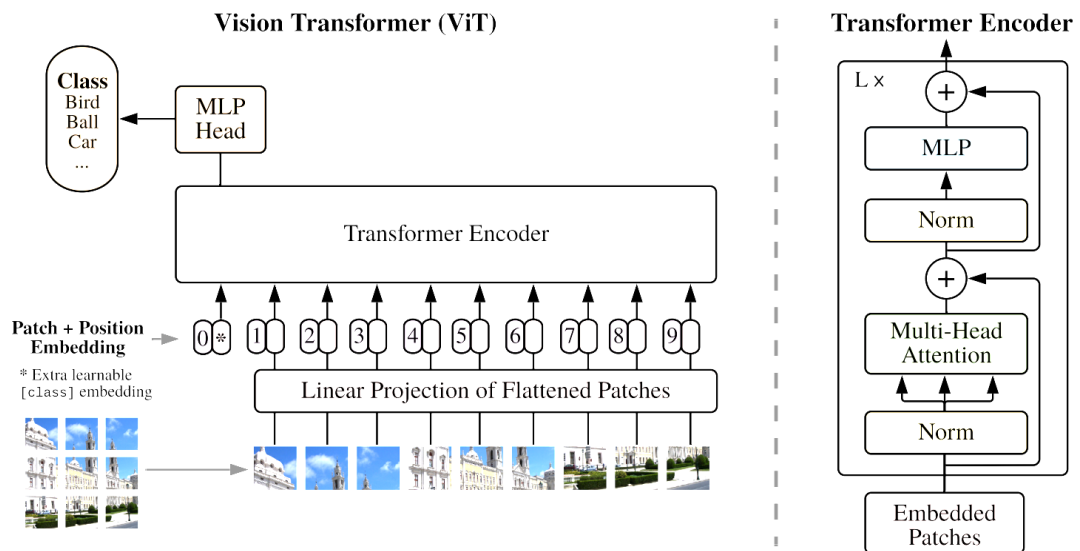
Neste capítulo, apresentam-se os conceitos fundamentais para a compreensão da otimização de kernels de inferência. A discussão abrange desde a arquitetura dos Vision Transformers até as minúcias da hierarquia de memória em Unidades de Processamento Gráfico (GPUs), culminando nas ferramentas de programação de baixo nível.

2.1 Vision Transformers e a Mecânica de Atenção

Os Vision Transformers (ViT) representam uma mudança de paradigma na visão computacional ao transpor a arquitetura de autoatenção, originalmente concebida para processamento de linguagem natural (Vaswani *et al.*, 2017), para o domínio de imagens. Segundo Dosovitskiy *et al.* (2021), o ViT decompõe uma imagem em patches fixos, os quais são tratados como sequências de tokens, permitindo que o modelo aprenda relações globais entre diferentes regiões da cena desde as camadas iniciais.

Como referência visual da arquitetura, a Figura 1 apresenta o fluxo do ViT, desde a tokenização em patches até o codificador Transformer e a cabeça de classificação.

Figura 1 – Arquitetura do Vision Transformer (ViT)



Fonte: Adaptado de Dosovitskiy *et al.* (2021).

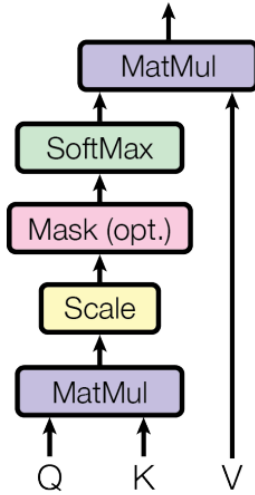
O cerne dessa arquitetura é o mecanismo de autoatenção, cujos componentes fundamentais, o Scaled Dot-Product Attention e sua extensão para o Multi-Head Attention,

são ilustrados na Figura 2. Matematicamente, a operação de atenção é definida pela Equação 2.1:

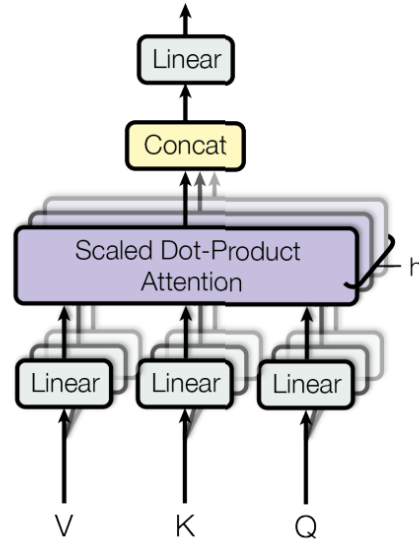
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (2.1)$$

Figura 2 – Mecanismos de atenção do Transformer

(a) Scaled Dot-Product Attention



(b) Multi-Head Attention



Fonte: Vaswani *et al.* (2017).

Onde Q , K e V representam as matrizes de Query, Key e Value. Embora eficaz, essa operação apresenta uma complexidade computacional $O(N^2)$, onde N é o número de patches. Como apontam Dao *et al.* (2022), o gargalo dessa operação em GPUs modernas não é o poder computacional bruto, mas sim a latência de memória gerada pela leitura e escrita de matrizes intermediárias de atenção.

2.1.1 Por que Autoatenção?

No trabalho original dos Transformers, Vaswani *et al.* (2017) justificam o uso da autoatenção comparando-a com camadas recorrentes e convolucionais para o mapeamento de uma sequência de entrada em outra de mesmo comprimento. A análise é guiada por três critérios centrais: (i) complexidade computacional por camada, (ii) grau de paralelização e (iii) comprimento do caminho entre dependências de longo alcance.

Sob essa ótica, a autoatenção apresenta uma vantagem estrutural importante: todas as posições da sequência interagem diretamente em uma única camada, com número constante de operações sequenciais. Em contraste, arquiteturas recorrentes exigem de-

pendência temporal passo a passo, com custo sequencial proporcional ao comprimento da sequência. Na prática, isso reduz o paralelismo possível em GPU e limita o throughput.

Também no aspecto de dependências de longo alcance, a autoatenção encurta o caminho de propagação de informação entre dois tokens para ordem constante. Em arquiteturas recorrentes, esse caminho cresce linearmente com o tamanho da sequência; em convolucionais, depende da largura do kernel e da profundidade da pilha de camadas. Essa diferença é relevante porque caminhos menores tendem a facilitar o aprendizado de relações globais.

Em termos de custo assintótico, a autoatenção densa envolve tipicamente $O(n^2d)$, enquanto camadas recorrentes escalam como $O(nd^2)$ e convoluções dependem de k (largura do kernel). Conforme discutido por Vaswani *et al.* (2017), quando $n < d$, cenário comum em representações de linguagem e compatível com várias configurações de tokens em visão, a autoatenção tende a ser computacionalmente mais favorável que abordagens recorrentes, além de ser mais paralelizável.

Por fim, os autores destacam que versões restritas de autoatenção (janela local de tamanho r) podem reduzir custo para sequências muito longas, ao custo de aumentar o caminho máximo para $O(n/r)$. Esse compromisso custo–alcance é particularmente útil para conectar a formulação teórica da atenção com o problema de engenharia tratado neste trabalho: minimizar movimentação de dados e latência de memória sem perder capacidade de modelar dependências globais.

2.2 Complexidade Computacional e Movimentação de Dados na Atenção

A Equação 2.1 evidencia que a atenção densa é composta por três etapas principais: o produto QK^T , a normalização por *softmax* e a multiplicação da matriz de probabilidades por V . Considerando uma cabeça de atenção com N tokens e dimensão D , o produto QK^T produz uma matriz de scores de dimensão $N \times N$ e possui custo aproximado de $2N^2D$ operações de ponto flutuante. A multiplicação final por V possui custo da mesma ordem, também próximo de $2N^2D$. Assim, desconsiderando constantes associadas ao *softmax*, o custo aritmético do forward de uma cabeça é da ordem de $O(N^2D)$.

O custo aritmético, entretanto, não descreve completamente o desempenho em GPU. Em uma implementação ingênua, a matriz QK^T é escrita na memória global, posteriormente lida para a aplicação do *softmax* e usada na multiplicação por V . Esse fluxo materializa um tensor intermediário de tamanho $N \times N$ que pode dominar o tráfego em HBM quando N cresce. Em ViTs, esse efeito aparece de forma direta: dobrar a resolução espacial ou reduzir o tamanho dos patches aumenta o número de tokens e amplia quadraticamente a matriz de atenção.

Para uma configuração com B amostras, H cabeças, N tokens e dimensão de cabeça

D , o custo aritmético aproximado do forward de atenção densa pode ser representado por:

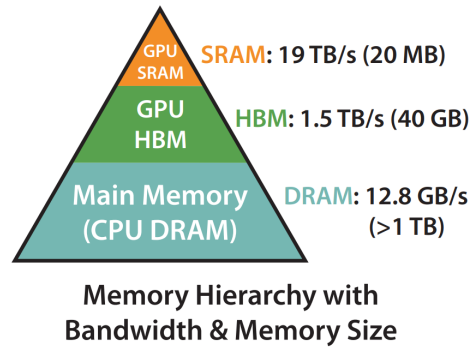
$$FLOPs \approx 4BHN^2D. \quad (2.2)$$

Já a matriz intermediária de scores ocupa, sozinha, BHN^2 elementos. Em FP16, isso corresponde a aproximadamente $2BHN^2$ bytes por materialização. Como uma implementação não fundida pode escrever e reler essa estrutura em mais de uma etapa, o tráfego efetivo de memória pode superar o tamanho dos tensores de entrada e saída. Essa diferença explica por que otimizações como FlashAttention e kernels fundidos não buscam apenas reduzir FLOPs, mas reduzir operações de entrada e saída na memória global (Dao *et al.*, 2022).

2.3 Arquitetura de GPU e a “Parede de Memória” (Memory Wall)

Para entender a necessidade de otimização de baixo nível, é preciso distinguir os diferentes níveis de memória em uma GPU. A memória principal, conhecida como High Bandwidth Memory (HBM), possui alta capacidade, porém latência significativamente superior à memória estática interna do chip, a SRAM.

Figura 3 – Hierarquia de memória com largura de banda e capacidade



Fonte: Adaptado de Dao *et al.* (2022).

O desequilíbrio entre a velocidade de processamento e a velocidade de transferência de dados é o que a literatura denomina como Memory Wall. Sobre este desafio, Wulf e McKee (1996) observam:

[...] a disparidade entre a velocidade dos processadores e a velocidade da memória principal tem crescido exponencialmente. Se essa tendência continuar, a performance dos sistemas será limitada não pela frequência de clock do processador, mas pela taxa na qual os dados podem ser entregues a ele (Wulf; McKee, 1996) (tradução do autor).

Dessa forma, otimizar um kernel significa maximizar o reuso dos dados em SRAM, registradores ou memória compartilhada, minimizando acessos à HBM. Para a atenção, essa ideia complementa a análise da seção anterior: não basta executar a mesma equação, é preciso organizar o fluxo de dados para evitar escritas e leituras intermediárias que degradam o throughput.

2.3.1 Modelo de execução em GPUs

Uma GPU executa milhares de threads organizadas em blocos, que são escalonados nos Streaming Multiprocessors. Cada SM possui recursos limitados de registradores, memória compartilhada, unidades de execução e slots para warps residentes. O desempenho de um kernel depende, portanto, de uma combinação entre paralelismo, ocupação, coalescência de acessos à memória e reutilização local de dados.

Threads são agrupadas em warps, que executam a mesma instrução sobre dados distintos. Quando os acessos à memória global são contíguos e alinhados, a GPU consegue agrupá-los em transações mais eficientes. Por outro lado, acessos irregulares, baixa reutilização de dados ou excesso de registradores por thread podem reduzir a ocupação e aumentar a exposição à latência de memória. Em kernels de atenção, esses fatores aparecem na escolha do tamanho dos blocos de Q , K e V , no número de warps por programa e na quantidade de dados mantida em registradores ou memória compartilhada.

Para o presente trabalho, essa visão é importante porque o objetivo não é apenas reescrever uma equação matemática em outra linguagem. O objetivo é organizar a mesma computação de modo que cada bloco carregado da HBM seja reutilizado tantas vezes quanto possível antes de ser descartado. A eficiência do kernel depende, assim, da relação entre trabalho aritmético realizado e bytes transferidos pela hierarquia de memória.

2.4 Modelo Roofline para Análise de Desempenho

O Modelo Roofline é uma ferramenta de análise que relaciona o desempenho atingível de uma aplicação com sua intensidade aritmética, isto é, a razão entre operações de ponto flutuante e bytes movimentados na memória (Williams; Waterman; Patterson, 2009). Nesse modelo, o desempenho máximo de um kernel é limitado por dois tetos principais: a largura de banda de memória e o pico computacional do hardware.

A intensidade aritmética pode ser escrita como:

$$I = \frac{FLOPs}{Bytes}. \quad (2.3)$$

Quando I é baixo, o kernel tende a ser *memory-bound*: seu desempenho é limitado pela taxa com que dados são fornecidos ao processador. Quando I é alto, o kernel tende a ser *compute-bound*: a limitação passa a ser a capacidade das unidades aritméticas. A

fronteira entre esses regimes ocorre no ponto em que o produto entre intensidade aritmética e largura de banda de memória atinge o pico computacional do dispositivo.

No contexto da atenção em ViTs, o Modelo Roofline permite avaliar se uma otimização melhora o desempenho por aumentar o uso das unidades aritméticas ou por reduzir o tráfego de memória. Um kernel fundido pode deslocar a operação no gráfico Roofline ao evitar a materialização da matriz $N \times N$, aumentando a intensidade aritmética efetiva e reduzindo a dependência da HBM. Por esse motivo, o modelo é adotado neste trabalho como ferramenta complementar às métricas de latência e throughput.

2.5 OpenAI Triton

Historicamente, a escrita de kernels otimizados exigia o uso de CUDA C++, uma linguagem de alta complexidade que demanda gerenciamento manual de threads e sincronização de blocos. O OpenAI Triton surge como uma alternativa de abstração que permite a escrita de kernels altamente performáticos utilizando uma sintaxe baseada em Python.

De acordo com Tillet, Kung e Cox (2019), o Triton introduz um paradigma baseado em blocos (tiles), onde o compilador é responsável por otimizar automaticamente o agendamento de tarefas e o uso dos registradores, permitindo que o desenvolvedor foque na lógica de movimentação de dados na hierarquia de memória.

Na prática, um kernel Triton é escrito como um programa parametrizado por blocos de dados. Cada instância do programa identifica sua região de atuação por meio de índices como `program_id`, carrega blocos com `tl.load`, executa operações vetoriais ou matriciais com primitivas como `tl.dot` e grava o resultado com `tl.store`. Máscaras booleanas são utilizadas para tratar bordas quando as dimensões do tensor não são múltiplas exatas do tamanho do bloco.

Parâmetros como `BLOCK_M`, `BLOCK_N`, `num_warps` e `num_stages` controlam a granularidade do trabalho e influenciam ocupação, pressão de registradores e paralelismo. Assim, o Triton oferece uma abstração intermediária: mais próxima da hierarquia de memória do que APIs de alto nível, mas menos verbosa que CUDA C++. Essa característica o torna adequado para o objetivo do trabalho, que é experimentar uma organização de dados inspirada em FlashAttention sem deslocar o foco para detalhes excessivos de sincronização manual de threads.

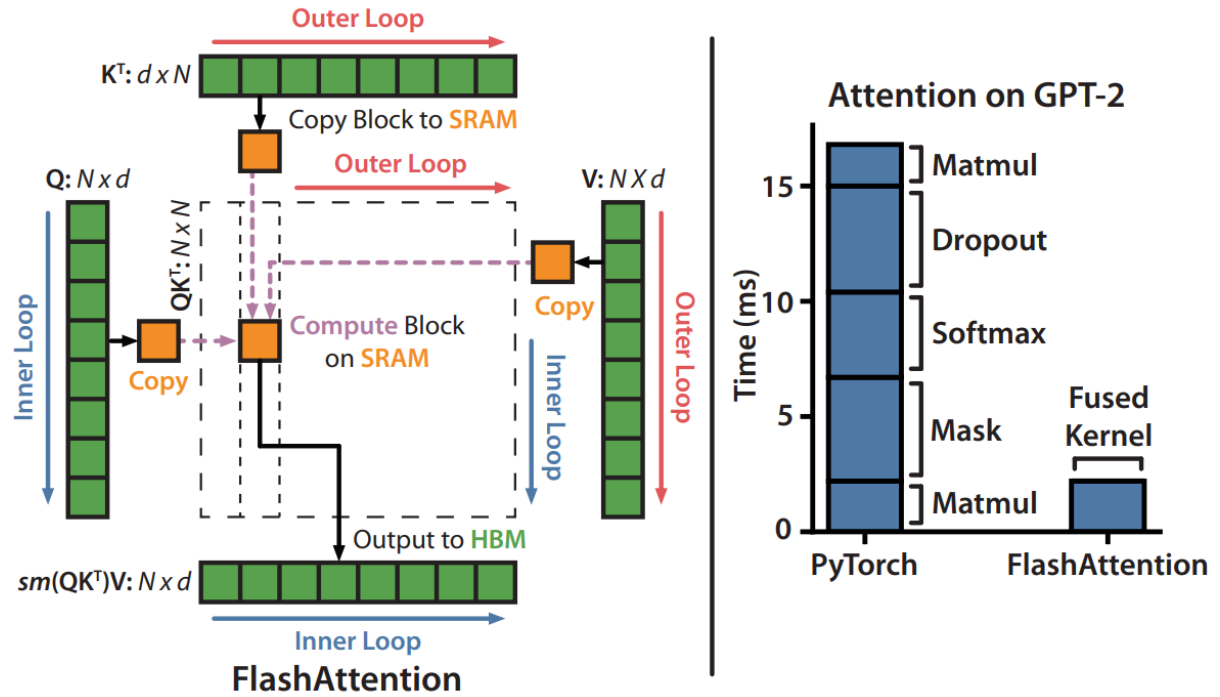
2.6 Fusão de Operadores e Flash Attention

A técnica de fusão de operadores (Operator Fusion) consiste em agrupar múltiplas operações matemáticas em um único kernel executável. No contexto de modelos de visão,

isso significa realizar o cálculo do Softmax e a multiplicação por V enquanto os dados de Q ainda residem nos registradores da GPU.

Como referência visual do método, a Figura 4 ilustra a hierarquia de memória, o particionamento em blocos e o ganho de desempenho obtido pelo kernel fundido do FlashAttention.

Figura 4 – Esquema do FlashAttention e comparação de tempo de execução



Fonte: Adaptado de Dao *et al.* (2022).

O FlashAttention explora essa ideia ao reorganizar o cálculo da atenção em blocos e evitar a materialização da matriz de atenção $N \times N$ na memória principal. Como sintetizado por Dao *et al.* (2022), essa organização reduz acessos à HBM e produz acelerações significativas em relação a implementações tradicionais, com menor consumo de memória.

3 METODOLOGIA

3.1 Ambiente Experimental

O ambiente experimental deste trabalho foi definido com o objetivo de isolar o impacto da implementação do kernel sobre a latência da operação de atenção. Os experimentos foram executados em uma GPU NVIDIA compatível com CUDA, utilizando PyTorch como baseline de alto nível e OpenAI Triton como ferramenta de desenvolvimento do kernel customizado. Para garantir reprodutibilidade, cada execução registrou o modelo da GPU, quantidade de SMs, versão do driver, versão do CUDA, versão do PyTorch, versão do Triton, tipo numérico utilizado e parâmetros de forma dos tensores.

Os tensores de entrada são organizados no formato $[B, H, N, D]$, em que B representa o tamanho do lote, H o número de cabeças de atenção, N o número de tokens e D a dimensão de cada cabeça. Esse formato é compatível com implementações usuais de Multi-Head Attention em modelos ViT. Adota-se $B = 1$, refletindo o cenário de inferência embarcada em tempo real, no qual cada quadro capturado pela câmera é processado individualmente, sem acúmulo de lote. Para o estudo de escala, consideram-se comprimentos alinhados $N = 128$, $N = 256$, $N = 512$, $N = 768$ e $N = 1024$: múltiplos de 32 são compatíveis com granularidades frequentes de kernels em GPU e permitem observar o crescimento quadrático da atenção sem misturar o efeito de bordas irregulares. Como esses valores não correspondem a um modelo real específico, avalia-se adicionalmente $N = 197$, comprimento de sequência do ViT-B/16 com entrada 224×224 (196 patches mais o token de classe) (Dosovitskiy *et al.*, 2021), representativo do caso de uso prático e não múltiplo do tamanho de bloco do kernel, o que também exercita o tratamento de bordas por máscaras.

As medições de tempo foram feitas com eventos CUDA, após uma etapa de aquecimento destinada a amortizar efeitos de compilação JIT, inicialização de contexto e variações transientes de frequência. O perfil de baixo nível foi coletado com NVIDIA Nsight Compute, registrando métricas de tráfego em memória global, ocupação, utilização de SMs e throughput computacional.

3.2 Design do Kernel Customizado

O kernel customizado implementa a etapa forward de atenção densa não causal, sem dropout, voltada ao cenário de inferência. O cálculo corresponde à Equação 2.1, porém evita a materialização completa da matriz QK^T , de dimensão $N \times N$, na memória global. A operação é decomposta em blocos de linhas de Q e blocos de colunas de K e V , permitindo que cada programa Triton carregue apenas uma fração da sequência por vez.

A estratégia de tiling utiliza dois parâmetros principais: $BLOCK_M$, que define a quantidade de queries processadas por programa, e $BLOCK_N$, que define a quantidade de keys e values carregados a cada iteração. Para cada bloco de queries, o kernel percorre a sequência de keys em blocos sucessivos, acumulando o resultado parcial da atenção. Desse modo, os valores intermediários necessários ao softmax permanecem em registradores ou SRAM durante a execução do programa, reduzindo leituras e escritas na HBM.

O escopo inicial da implementação considera tensores contíguos com eixos de lote, cabeças, sequência e dimensão por cabeça, além de dimensões de cabeça até 128 e tipos numéricos de baixa precisão, como FP16 e BF16. Essa delimitação é compatível com o objetivo experimental do trabalho: comparar o impacto da fusão de operadores e da consciência da hierarquia de memória em um caso representativo de atenção em Vision Transformers.

3.3 Implementação do Softmax Online

A comparação detalhada entre uma implementação padrão de *softmax* em PyTorch e uma versão otimizada com OpenAI Triton é apresentada no Apêndice A. O apêndice inclui os trechos de código e uma análise objetiva dos impactos em movimentação de memória, paralelismo e ocupação do hardware.

No kernel de atenção, entretanto, o *softmax* não é computado como uma operação isolada. Em vez de armazenar a matriz de scores completa, utiliza-se a formulação de *softmax online*, proposta originalmente por Milakov e Gimelshein (2018) e empregada pelo FlashAttention (Dao *et al.*, 2022). Para cada linha da matriz de atenção, o kernel mantém três acumuladores: o maior valor observado m_i , a soma normalizada l_i e o acumulador vetorial de saída O_i .

Ao processar um novo bloco de scores S_{ij} , o máximo da linha é atualizado de forma incremental:

$$m_i^{novo} = \max \left(m_i^{antigo}, \max_j S_{ij} \right). \quad (3.1)$$

Em seguida, a soma exponencial anterior é reescalada para a nova referência numérica:

$$l_i^{novo} = e^{m_i^{antigo} - m_i^{novo}} l_i^{antigo} + \sum_j e^{S_{ij} - m_i^{novo}}. \quad (3.2)$$

O acumulador da saída passa pelo mesmo reescalonamento:

$$O_i^{novo} = e^{m_i^{antigo} - m_i^{novo}} O_i^{antigo} + \sum_j e^{S_{ij} - m_i^{novo}} V_j. \quad (3.3)$$

Ao final da varredura sobre todos os blocos de keys e values, a saída final é obtida por O_i/l_i . Essa formulação preserva a estabilidade numérica do *softmax* tradicional, pois continua subtraindo o maior valor da linha, mas permite que a normalização seja realizada em fluxo, sem escrita da matriz intermediária de atenção na memória global.

3.4 Protocolo de Avaliação

O protocolo de avaliação compara três implementações: uma versão explícita em PyTorch, que materializa as etapas QK^T , *softmax* e multiplicação por V ; a função otimizada `scaled_dot_product_attention` (SDPA) do PyTorch; e o kernel customizado em Triton. A primeira serve como referência de corretude e como baseline de alto nível mais transparente. A segunda representa uma implementação de biblioteca otimizada: no ambiente avaliado, verificou-se por meio do `torch.profiler` que a chamada é despachada para o kernel `pytorch_flash::flash_fwd_kernel`, correspondente à implementação do FlashAttention-2 (Dao, 2023) integrada ao PyTorch. O kernel customizado, portanto, é comparado diretamente com uma implementação representativa do estado da arte. A terceira implementação corresponde à contribuição experimental deste trabalho.

Na implementação explícita, o *softmax* é calculado em FP32 por estabilidade numérica, o que materializa a matriz $N \times N$ em precisão simples (4 bytes por elemento) e amplia o tráfego de memória do baseline em relação à estimativa de $2BHN^2$ bytes apresentada para FP16 na seção de complexidade computacional. Essa escolha é registrada explicitamente porque afeta a leitura dos volumes de DRAM medidos no baseline eager.

Para cada configuração de B , H , N , D e tipo numérico, são gerados tensores aleatórios em GPU e executadas rodadas de aquecimento antes das medições. Em cada execução, a latência média é calculada a partir de 100 iterações sincronizadas por eventos CUDA, precedidas de 25 iterações de aquecimento. Como a GPU avaliada não opera com frequências travadas e compartilha o dispositivo com a interface gráfica do sistema, cada configuração foi medida em cinco execuções independentes, reportando-se a mediana, de modo a reduzir a influência de variações transientes de clock e de carga. A corretude é verificada pelo erro absoluto máximo entre a saída do kernel avaliado e a implementação explícita em PyTorch.

Além da latência, calcula-se o throughput aproximado em TFLOP/s a partir da estimativa de custo aritmético apresentada na seção de complexidade computacional, seguindo a convenção usual de desconsiderar as operações do *softmax* (Dao *et al.*, 2022). Essa estimativa permite comparar as implementações sob a mesma escala de trabalho. A interpretação final combina latência, throughput, tráfego de memória global e Modelo Roofline, distinguindo casos *memory-bound* de casos *compute-bound*. Cabe observar que as métricas de perfil coletadas quantificam o tráfego em DRAM; a eficiência do uso de SRAM é avaliada de forma indireta, pela redução desse tráfego, uma vez que dados reutilizados

em registradores e SRAM deixam de ser relidos da memória global.

4 RESULTADOS E DISCUSSÕES

4.1 Análise Comparativa de Latência

Os experimentos foram executados em uma GPU NVIDIA GeForce RTX 5060 com 30 SMs, utilizando Python 3.13.12, PyTorch 2.11.0 com CUDA 13.0 e tensores em FP16. A configuração avaliada corresponde a $B = 1$, $H = 12$ e $D = 64$. A Tabela 1 resume pontos alinhados e usuais em avaliações de camadas Transformer, mantendo o número de tokens como múltiplo de 32, além do ponto $N = 197$, correspondente ao ViT-B/16 com entrada 224×224 . Esses valores representam uma progressão de sequências pequenas, médias e grandes para a operação de atenção. Em seguida, os gráficos apresentam uma varredura complementar com N variando de 32 a 1024 em passos de 32, permitindo observar a tendência de escala das implementações.

Tabela 1 – Resultados comparativos de latência por comprimento de sequência (mediana de cinco execuções independentes)

N	Eager (ms)	SDPA (ms)	Triton (ms)	Speedup SDPA	Speedup Triton
128	0,1367	0,0251	0,0332	$5,44\times$	$4,12\times$
197	0,1655	0,0255	0,0367	$6,49\times$	$4,51\times$
256	0,1333	0,0242	0,0373	$5,51\times$	$3,57\times$
512	0,1480	0,0316	0,0457	$4,68\times$	$3,24\times$
768	0,5096	0,0707	0,0997	$7,21\times$	$5,11\times$
1024	1,0003	0,1165	0,1615	$8,58\times$	$6,20\times$

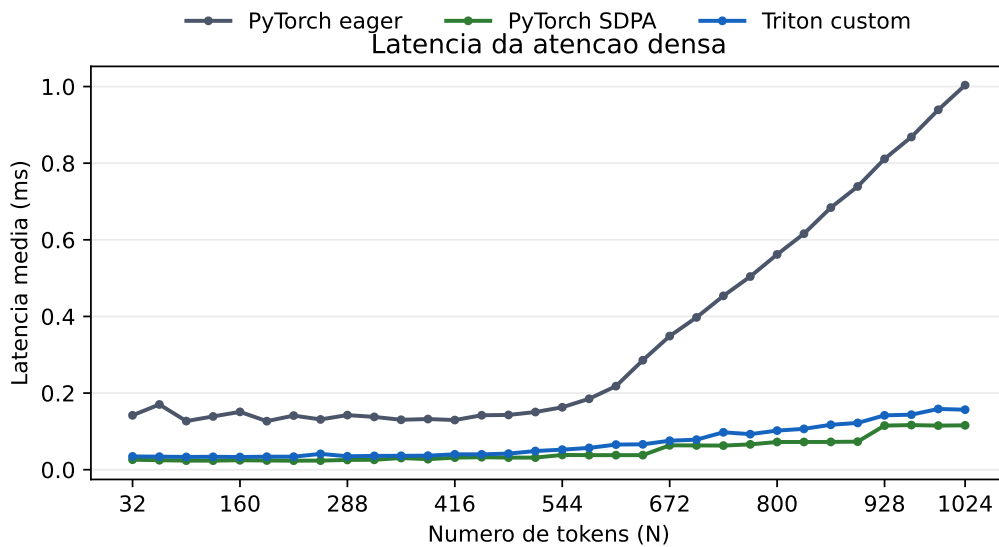
Fonte: Elaborada pelo autor.

O ponto $N = 197$ merece destaque por corresponder a uma configuração real de modelo, e não a um tamanho alinhado artificialmente. Nele, o kernel Triton mantém speedup de $4,51\times$ sobre o eager, da mesma ordem dos pontos alinhados vizinhos, o que indica que o tratamento de bordas por máscaras não degrada o desempenho quando N não é múltiplo do tamanho de bloco. Observa-se, inclusive, que a implementação eager é mais lenta em $N = 197$ do que em $N = 256$: dimensões não alinhadas penalizam a seleção de kernels das bibliotecas de álgebra linear usadas pela versão explícita, enquanto as implementações fundidas, que já tratam bordas por máscaras, mostram-se insensíveis ao desalinhamento. Nessa escala, entretanto, o custo fixo de lançamento de kernels domina a latência e os ganhos absolutos são modestos (cerca de 0,13 ms por camada de atenção); o benefício da fusão torna-se mais expressivo em resoluções de entrada maiores ou patches menores, que elevam N .

A Figura 5 mostra a evolução da latência ao longo da varredura regular de tokens. As implementações otimizadas permanecem abaixo da versão explícita em PyTorch em toda

a faixa analisada. Para tamanhos pequenos, o custo fixo de lançamento e escalonamento dos kernels tem peso relevante; por isso, a latência não cresce proporcionalmente a N^2 nos primeiros pontos. À medida que a sequência aumenta, o custo quadrático da atenção se torna mais visível, especialmente na implementação eager.

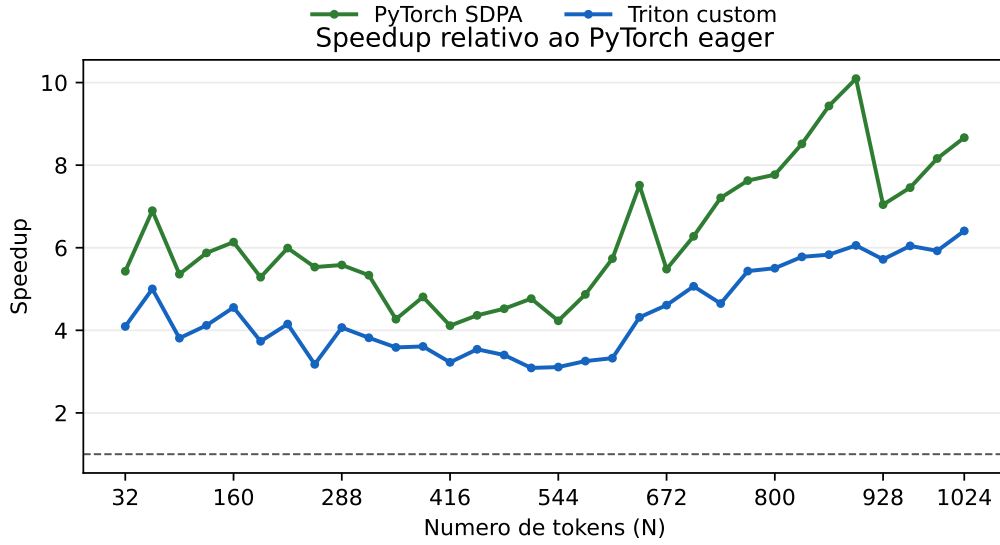
Figura 5 – Comparação de latência entre as implementações de atenção



Fonte: Elaborada pelo autor.

O speedup relativo ao eager, apresentado na Figura 6, mostra que o ganho das versões otimizadas aumenta com o tamanho da sequência. Na varredura, o SDPA chega a aproximadamente $8,67\times$ em $N = 1024$, enquanto o kernel Triton atinge aproximadamente $6,41\times$ no mesmo ponto. O Triton customizado, portanto, valida o benefício da fusão de operadores em relação à implementação explícita, mas fica abaixo do backend especializado do PyTorch na maior parte da faixa regular avaliada.

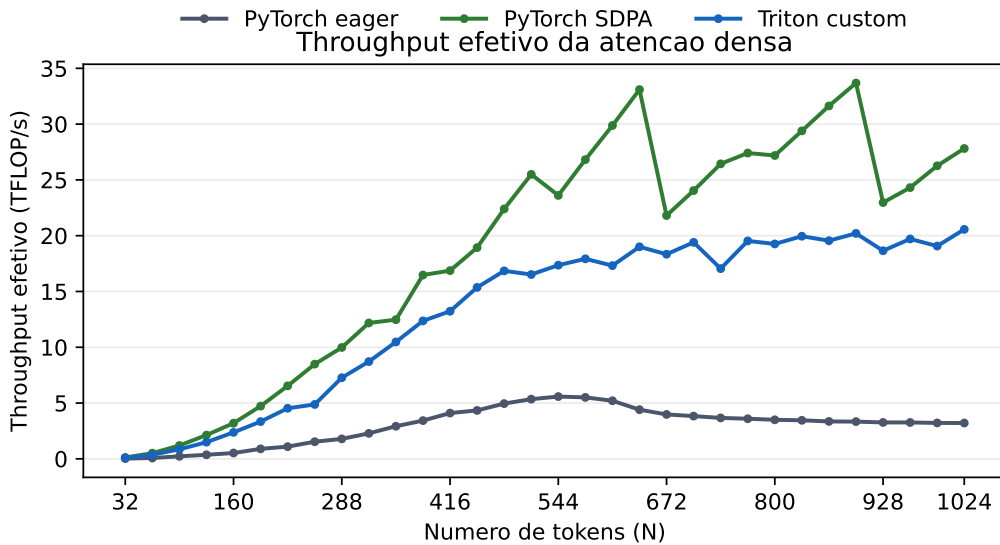
Figura 6 – Speedup relativo à implementação PyTorch eager



Fonte: Elaborada pelo autor.

O throughput efetivo, apresentado na Figura 7, reforça a mesma leitura. Na extremidade da varredura, em $N = 1024$, o SDPA alcança 27,82 TFLOP/s efetivos, enquanto o Triton atinge 20,56 TFLOP/s. Esses valores devem ser interpretados como throughput efetivo da operação de atenção, calculado a partir da estimativa $4BHN^2D$, e não como o pico teórico absoluto da GPU.

Figura 7 – Throughput efetivo das implementações de atenção



Fonte: Elaborada pelo autor.

Do ponto de vista de corretude numérica, os erros máximos observados ficaram na ordem de 10^{-4} a 10^{-3} para as implementações otimizadas em FP16. Essa diferença é

esperada, pois SDPA e Triton podem utilizar ordem de operações, acumulação, fusão e instruções internas diferentes da implementação explícita usada como referência. O erro nulo da versão eager decorre do fato de ela ser a própria referência de comparação.

Em conjunto, os resultados indicam que os pontos alinhados selecionados são úteis para resumir o comportamento em tamanhos usuais de sequência, enquanto a varredura completa em múltiplos de 32 é mais adequada para discutir comportamento assintótico, efeitos de alinhamento e estabilidade do ganho de desempenho.

4.2 Profiling de Baixo Nível

Os resultados sugerem que o kernel Triton desenvolvido neste trabalho explora corretamente a ideia de evitar a materialização da matriz de atenção, mas ainda não atinge o mesmo nível de otimização do backend usado pelo PyTorch SDPA no caso maior. Na implementação avaliada, os parâmetros principais do kernel são `BLOCK_M=16` e `BLOCK_N=64`. Para $N = 1024$, isso implica 64 blocos de queries e 16 blocos de keys/values por cabeça. Como cada bloco de queries percorre toda a sequência de keys, os blocos de K e V são recarregados várias vezes ao longo da execução.

Esse padrão é suficiente para demonstrar o benefício da fusão em relação ao eager, mas pode limitar a eficiência para sequências maiores. Mesmo usando comprimentos múltiplos de 32, a manutenção dos acumuladores do *softmax online* em FP32 aumenta a pressão sobre registradores, podendo reduzir ocupação dependendo da arquitetura e da configuração escolhida pelo compilador Triton.

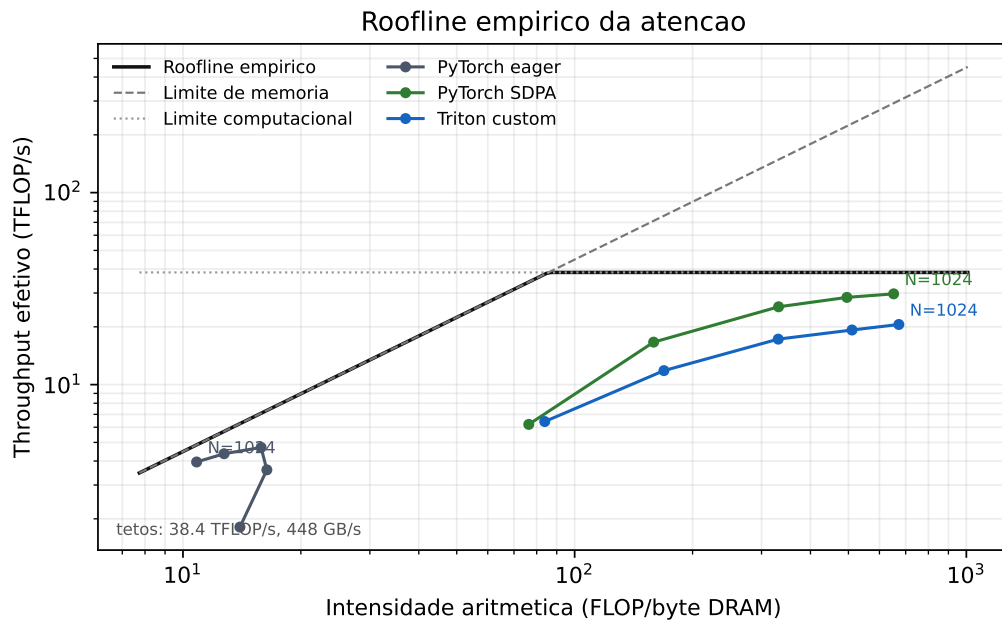
Na varredura com múltiplos de 32, os degraus observados nas curvas também são coerentes com essa estrutura de blocos. Embora todos os valores sejam múltiplos de `BLOCK_M=16`, apenas parte deles é múltipla de `BLOCK_N=64`; além disso, sempre que a sequência cruza um novo limite de bloco, aumenta o número de programas ou de iterações internas do kernel. Por isso, a curva de latência não precisa ser perfeitamente suave mesmo quando N cresce de forma regular.

Para complementar as medições de tempo, foi executado profiling com NVIDIA Nsight Compute por meio da interface `nsight-python`. Foram coletados os contadores `dram__bytes.sum`, `dram__bytes_op_read.sum`, `dram__bytes_op_write.sum` e `gpu__time__duration.sum`, permitindo estimar a intensidade aritmética efetiva e o throughput de cada implementação. Cada combinação de backend e número de tokens foi perfilada em um relatório separado, reduzindo interferências entre kernels de biblioteca e kernels gerados pelo Triton.

4.3 Modelo Roofline

A partir dos bytes medidos em DRAM e da estimativa de operações $4BHN^2D$, foi construído o gráfico Roofline da Figura 8. Os tetos exibidos correspondem às características da GPU avaliada: 448 GB/s de banda de memória GDDR7, valor nominal da RTX 5060, e aproximadamente 38 TFLOP/s de pico FP16 denso em Tensor Cores, estimado a partir das especificações oficiais do fabricante. O ponto de inflexão entre os regimes *memory-bound* e *compute-bound* da GPU ocorre, portanto, em torno de 86 FLOP/byte.

Figura 8 – Roofline empírico das implementações de atenção



Fonte: Elaborada pelo autor.

Em $N = 1024$, a implementação eager movimentou aproximadamente 297,3 MB em DRAM, com intensidade aritmética de 10,84 FLOP/byte e 3,95 TFLOP/s efetivos. Por estar bem abaixo do ponto de inflexão, o eager opera em regime claramente *memory-bound*: seu teto imposto pela banda de memória é de cerca de 4,9 TFLOP/s ($10,84 \times 448$ GB/s), do qual a implementação atinge aproximadamente 81%. No mesmo ponto, o SDPA movimentou aproximadamente 4,94 MB, alcançando 652,21 FLOP/byte e 29,70 TFLOP/s. O kernel Triton movimentou volume semelhante, aproximadamente 4,79 MB, e atingiu 672,37 FLOP/byte, mas permaneceu em 20,57 TFLOP/s. Ambas as implementações fundidas cruzam para o regime *compute-bound*: o SDPA opera em torno de 77% do pico FP16 estimado da GPU, enquanto o Triton alcança cerca de 54%. Essa diferença é importante: ela mostra que o Triton customizado de fato reduz a movimentação de memória global em relação ao eager, mas seu desempenho inferior ao SDPA não pode ser explicado apenas pelo tráfego de DRAM.

Assim, o Roofline reforça a hipótese central do trabalho e, ao mesmo tempo, delimita a contribuição experimental. A fusão de operadores aumenta substancialmente a intensidade aritmética ao evitar a materialização da matriz intermediária $N \times N$, mas a aproximação do teto de desempenho também depende de reutilização local dos blocos, ocupação efetiva, pressão de registradores, escalonamento de warps e parâmetros de tiling adequados à GPU avaliada. O kernel em Triton é, portanto, eficiente frente ao eager, enquanto o SDPA representa uma referência de biblioteca altamente otimizada.

4.4 Implicações para Inferência Embarcada em VANTs

Uma propriedade útil do Modelo Roofline é que a intensidade aritmética é uma característica do algoritmo e do fluxo de dados, não do hardware: os valores de FLOP/byte medidos nesta seção transferem-se diretamente para outras GPUs, enquanto os tetos de banda e de pico computacional mudam com a plataforma. Isso permite projetar, de forma analítica, o comportamento das três implementações no hardware embarcado típico de VANTs, mesmo sem repetir os experimentos nesses módulos.

Considere-se o NVIDIA Jetson Orin Nano, módulo representativo da classe de computadores embarcados utilizados em VANTs de pequeno porte, com cerca de 68 GB/s de banda de memória LPDDR5 compartilhada entre CPU e GPU e pico da ordem de 10 TFLOP/s em FP16 denso (NVIDIA Corporation, 2024). Com a intensidade aritmética medida de 10,84 FLOP/byte, a implementação eager ficaria limitada pela banda de memória a aproximadamente $10,84 \times 68 \approx 0,74$ TFLOP/s, menos de 8% do pico computacional do módulo, e isso no cenário otimista em que toda a banda estivesse disponível para a GPU. Já as implementações fundidas, com intensidade aritmética superior a 650 FLOP/byte, teriam teto de memória acima de 44 TFLOP/s, muito além do pico do módulo: sua limitação passaria a ser exclusivamente computacional. Em outras palavras, no hardware embarcado o kernel fundido pode, em princípio, explorar todo o potencial aritmético do dispositivo, enquanto a implementação ingênua fica confinada a uma fração pequena dele. Como a razão entre pico computacional e banda de memória é maior no módulo embarcado (cerca de 147 FLOP/byte no Orin Nano contra 86 FLOP/byte na RTX 5060 avaliada), o benefício relativo da fusão de operadores tende a ser maior no drone do que na estação de trabalho.

Há ainda uma implicação energética direta. O acesso à DRAM consome ordens de grandeza mais energia por byte do que operações aritméticas realizadas no chip (Horowitz, 2014). A redução de aproximadamente 297 MB para menos de 5 MB de tráfego em DRAM por operação de atenção, observada em $N = 1024$, representa portanto não apenas ganho de latência, mas redução do custo energético por inferência, fator que em um VANT se converte em autonomia de voo. A quantificação experimental desse ganho energético, por meio de medição de potência durante a inferência, é apontada como trabalho futuro.

Duas ressalvas delimitam essa análise. Primeiro, trata-se de uma projeção analítica: latências absolutas não são transferíveis entre plataformas, e efeitos como o compartilhamento da banda de memória com a CPU, limites térmicos e diferenças de arquitetura entre gerações de GPU só podem ser capturados por experimentos no módulo real, viáveis com a mesma pilha de software utilizada neste trabalho, uma vez que PyTorch e Triton são suportados no ambiente JetPack dos módulos Jetson. Segundo, a otimização em nível de kernel é complementar, e não concorrente, às abordagens em nível de arquitetura de modelo voltadas a dispositivos móveis, como MobileViT (Mehta; Rastegari, 2022) e EfficientViT (Cai *et al.*, 2023): um modelo compacto projetado para a borda também executa operações de atenção e igualmente se beneficia de kernels que evitem a materialização de tensores intermediários.

5 CONCLUSÃO

Este trabalho investigou a otimização de kernels de inferência para a operação de autoatenção em Vision Transformers, com foco na redução de acessos à memória global da GPU e nas implicações para inferência embarcada em VANTs. A partir de um kernel customizado em OpenAI Triton, foram exploradas técnicas de fusão de operadores, tiling e *softmax online*, mantendo o escopo restrito ao forward de atenção densa não causal em cenário de inferência.

Retomando os objetivos específicos definidos no Capítulo 1, todos foram cumpridos. O kernel fundido foi desenvolvido em Triton, integrando produto escalar, softmax e ponderação de valores em um único kernel, com estratégia de tiling parametrizada por blocos de queries e de keys/values. O profiling comparativo foi realizado contra dois baselines em PyTorch, a implementação explícita e o SDPA, cobrindo latência, throughput efetivo, erro numérico e tráfego de DRAM medido com NVIDIA Nsight Compute. A análise pelo Modelo Roofline, calibrada com os tetos nominais da GPU avaliada, identificou os gargalos remanescentes do kernel. Por fim, as intensidades aritméticas medidas foram utilizadas para projetar o comportamento das implementações em hardware embarcado representativo de VANTs.

Os resultados confirmam a hipótese principal do trabalho: evitar a materialização da matriz intermediária de atenção reduz substancialmente o tráfego em DRAM e melhora o desempenho em relação à implementação explícita em PyTorch. Na configuração avaliada, o kernel Triton atingiu speedup de $6,20\times$ sobre o eager em $N = 1024$, mantendo $4,51\times$ mesmo no caso realista $N = 197$ do ViT-B/16, enquanto o SDPA do PyTorch atingiu $8,58\times$. O profiling de baixo nível reforçou essa leitura: no ponto $N = 1024$, o eager movimentou cerca de 297,3 MB em DRAM, enquanto Triton e SDPA movimentaram aproximadamente 4,79 MB e 4,94 MB, respectivamente, cruzando do regime *memory-bound* para o *compute-bound*.

A comparação com o SDPA também delimita a contribuição do trabalho. Embora o kernel customizado reduza o tráfego de memória para um patamar semelhante ao da biblioteca otimizada, seu throughput permanece inferior, cerca de 54% do pico FP16 estimado da GPU contra 77% do SDPA, que no ambiente avaliado corresponde ao FlashAttention-2 integrado ao PyTorch. Isso indica que a eliminação de escritas intermediárias é condição importante, mas não suficiente, para aproximar a execução dos limites efetivos do hardware. Fatores como ocupação, pressão de registradores, reutilização local dos blocos, escalonamento de warps e escolha dos parâmetros de tiling continuam determinantes para o desempenho final.

Do ponto de vista da aplicação aeronáutica, a análise da Seção 4.4 indica que o benefício da fusão de operadores tende a ser ainda maior no hardware embarcado que equipa VANTs: em um módulo como o Jetson Orin Nano, a implementação ingênua ficaria confinada pela banda de memória a menos de 8% do pico computacional, enquanto o kernel fundido passaria ao regime *compute-bound*. A redução do tráfego de DRAM traduz-se, adicionalmente, em menor custo energético por inferência, fator diretamente ligado à autonomia de voo.

Assim, a contribuição central não é propor um kernel superior ao estado da arte, mas apresentar uma implementação experimental e um protocolo de análise capaz de explicar onde os ganhos ocorrem, quais gargalos permanecem e o que os resultados implicam para a percepção embarcada em sistemas aéreos autônomos. Como trabalhos futuros, destacam-se: a validação experimental em módulos embarcados da família Jetson, incluindo medição de potência e energia por inferência; a integração do kernel em um modelo ViT completo, medindo o impacto na latência de ponta a ponta da inferência; a aplicação de autotuning aos parâmetros do kernel; a avaliação em diferentes GPUs e tipos numéricos; a inclusão de atenção causal; e a extensão para treinamento, incluindo a etapa backward.

REFERÊNCIAS

- CAI, H. *et al.* Efficientvit: Multi-scale linear attention for high-resolution dense prediction. *In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. [S.l.: s.n.], 2023. Disponível em: <https://arxiv.org/abs/2205.14756>.
- DAO, T. **FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning**. 2023. Disponível em: <https://arxiv.org/abs/2307.08691>.
- DAO, T. *et al.* **FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness**. 2022. Disponível em: <https://arxiv.org/abs/2205.14135>.
- DOSOVITSKIY, A. *et al.* An image is worth 16x16 words: Transformers for image recognition at scale. *In: International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2021. Disponível em: <https://arxiv.org/abs/2010.11929>.
- HOROWITZ, M. 1.1 computing's energy problem (and what we can do about it). *In: 2014 IEEE International Solid-State Circuits Conference (ISSCC)*. [S.l.: s.n.], 2014. p. 10–14.
- KAUFMANN, E. *et al.* Champion-level drone racing using deep reinforcement learning. *Nature*, v. 620, n. 7976, p. 982–987, 2023.
- LOQUERCIO, A. *et al.* Learning high-speed flight in the wild. *Science Robotics*, v. 6, n. 59, p. eabg5810, 2021.
- MEHTA, S.; RASTEGARI, M. Mobilevit: Light-weight, general-purpose, and mobile-friendly vision transformer. *In: International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2022. Disponível em: <https://arxiv.org/abs/2110.02178>.
- MILAKOV, M.; GIMELSHEIN, N. **Online normalizer calculation for softmax**. 2018. Disponível em: <https://arxiv.org/abs/1805.02867>.
- NVIDIA Corporation. **NVIDIA Jetson Orin Nano Series Modules Data Sheet**. 2024. Acesso em: 10 jul. 2026. Disponível em: <https://developer.nvidia.com/embedded/jetson-orin>.
- OpenAI. **Triton documentation: Fused Softmax tutorial**. 2025. Acesso em: 10 jul. 2026. Disponível em: <https://triton-lang.org/main/getting-started/tutorials/02-fused-softmax.html>.
- TILLET, P.; KUNG, H. T.; COX, D. Triton: an intermediate language and compiler for tiled neural network computations. *In: Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. New York, NY, USA: Association for Computing Machinery, 2019. (MAPL 2019), p. 10–19. ISBN 9781450367196. Disponível em: <https://doi.org/10.1145/3315508.3329973>.
- VASWANI, A. *et al.* Attention is all you need. *In: Advances in Neural Information Processing Systems 30 (NIPS 2017)*. Long Beach, CA, USA: [S.l.: s.n.], 2017. Disponível em: <https://arxiv.org/abs/1706.03762>.

WILLIAMS, S.; WATERMAN, A.; PATTERSON, D. Roofline: an insightful visual performance model for multicore architectures. **Communications of the ACM**, Association for Computing Machinery, New York, NY, USA, v. 52, n. 4, p. 65–76, 04 2009. ISSN 0001-0782. Disponível em: <https://doi.org/10.1145/1498765.1498785>.

WULF, W.; MCKEE, S. Hitting the memory wall: Implications of the obvious. **Computer Architecture News**, v. 23, 01 1996.

APÊNDICES

APÊNDICE A – COMPARAÇÃO ENTRE SOFTMAX PADRÃO E SOFTMAX COM TRITON

Este apêndice apresenta uma comparação entre duas abordagens para cálculo de *softmax* em linhas de uma matriz: uma implementação padrão em PyTorch e uma implementação otimizada com OpenAI Triton. As implementações apresentadas a seguir são adaptadas do tutorial oficial *Fused Softmax* da documentação do OpenAI Triton (OpenAI, 2025), e são reproduzidas aqui como material didático de apoio à discussão do Capítulo 3.

A.1 Implementação padrão em PyTorch

```
import torch

import triton
import triton.language as tl
from triton.runtime import driver

DEVICE = triton.runtime.driver.active.get_active_torch_device()

def is_hip():
    return (
        triton.runtime.driver.active.get_current_target().backend == "hip"
    )

def is_cdna():
    return (
        is_hip()
        and triton.runtime.driver.active.get_current_target().arch in (
            'gfx940', 'gfx941', 'gfx942', 'gfx90a', 'gfx908'
        )
    )

def naive_softmax(x):
    """Compute row-wise softmax of X using native pytorch
```

We subtract the maximum element in order to avoid overflows. Softmax is invariant to this shift.

```
"""
# read MN elements ; write M elements
x_max = x.max(dim=1)[0]
# read MN + M elements ; write MN elements
z = x - x_max[:, None]
# read MN elements ; write MN elements
numerator = torch.exp(z)
# read MN elements ; write M elements
denominator = numerator.sum(dim=1)
# read MN + M elements ; write MN elements
ret = numerator / denominator[:, None]
# in total: read 5MN + 2M elements ; wrote 3MN + 2M elements
return ret
```

A.2 Implementação otimizada com Triton

O kernel abaixo processa uma linha inteira da matriz por programa, mantendo-a em SRAM durante todo o cálculo, de modo que cada elemento é lido da memória global uma única vez:

```
@triton.jit
def softmax_kernel(output_ptr, input_ptr, input_row_stride,
                  output_row_stride, n_rows, n_cols,
                  BLOCK_SIZE: tl.constexpr,
                  num_stages: tl.constexpr):
    # starting row of the program
    row_start = tl.program_id(0)
    row_step = tl.num_programs(0)
    for row_idx in tl.range(row_start, n_rows, row_step,
                           num_stages=num_stages):
        # The stride represents how much we need to increase the
        # pointer to advance 1 row
        row_start_ptr = input_ptr + row_idx * input_row_stride
        # The block size is the next power of two greater than
        # n_cols, so we can fit each row in a single block
        col_offsets = tl.arange(0, BLOCK_SIZE)
        input_ptrs = row_start_ptr + col_offsets
```

```

# Load the row into SRAM, using a mask since BLOCK_SIZE
# may be greater than n_cols
mask = col_offsets < n_cols
row = tl.load(input_ptrs, mask=mask, other=-float('inf'))
# Subtract maximum for numerical stability
row_minus_max = row - tl.max(row, axis=0)
numerator = tl.exp(row_minus_max)
denominator = tl.sum(numerator, axis=0)
softmax_output = numerator / denominator
# Write back output to DRAM
output_ptrs = (output_ptr + row_idx * output_row_stride
               + col_offsets)
tl.store(output_ptrs, softmax_output, mask=mask)

```

A função auxiliar a seguir consulta as propriedades do dispositivo para dimensionar o número de programas persistentes lançados:

```

properties = driver.active.utils.get_device_properties(DEVICE.index)
NUM_SM = properties["multiprocessor_count"]
NUM_REGS = properties["max_num_regs"]
SIZE_SMEM = properties["max_shared_mem"]
WARP_SIZE = properties["warpSize"]
target = triton.runtime.driver.active.get_current_target()
kernels = {}

```

```

def softmax(x):
    n_rows, n_cols = x.shape

    # The block size of each loop iteration is the smallest power of two
    # greater than the number of columns in 'x'
    BLOCK_SIZE = triton.next_power_of_2(n_cols)

    # Another trick we can use is to ask the compiler to use more threads
    # per row by increasing the number of warps ('num_warps')
    # over which each row is distributed.
    num_warps = 8

    # Number of software pipelining stages.
    num_stages = 4 if SIZE_SMEM > 200000 else 2

```

```
# Allocate output
y = torch.empty_like(x)

# pre-compile kernel to get register usage
# and compute thread occupancy.
kernel = softmax_kernel.warmup(
    y, x, x.stride(0), y.stride(0), n_rows, n_cols,
    BLOCK_SIZE=BLOCK_SIZE, num_stages=num_stages,
    num_warps=num_warps, grid=(1, )
)
kernel._init_handles()
n_regs = kernel.n_regs
size_smem = kernel.metadata.shared

if is_hip():
    # NUM_REGS represents the number of regular purpose registers.
    # On CDNA architectures this is half of all registers available.
    NUM_GPRS = NUM_REGS
    if is_cdna():
        NUM_GPRS = NUM_REGS * 2

    # MAX_NUM_THREADS represents maximum resident threads per SM.
    MAX_NUM_THREADS = properties["max_threads_per_sm"]
    max_num_waves = MAX_NUM_THREADS // WARP_SIZE
    occupancy = min(NUM_GPRS // WARP_SIZE // n_regs, max_num_waves)
    occupancy = occupancy // num_warps
else:
    occupancy = NUM_REGS // (n_regs * WARP_SIZE * num_warps)

occupancy = min(occupancy, SIZE_SMEM // size_smem)
num_programs = NUM_SM * occupancy
num_programs = min(num_programs, n_rows)

# Create a number of persistent programs.
kernel[(num_programs, 1, 1)](
    y, x, x.stride(0), y.stride(0), n_rows, n_cols,
    BLOCK_SIZE, num_stages
)
```

```
return y
```

A.3 Comparação técnica

- **Movimentação de memória:** a versão padrão materializa tensores intermediários para cada etapa; a versão Triton busca reduzir leituras e escritas na HBM por meio de execução em blocos e reutilização de dados em SRAM.
- **Paralelismo:** na implementação Triton, parâmetros como `BLOCK_SIZE`, `num_warps` e `num_stages` são ajustados para aumentar ocupação e throughput.
- **Consciência de hardware:** o código Triton utiliza propriedades do dispositivo (SMs, registradores e memória compartilhada) para definir o número de programas persistentes, aproximando a implementação do limite arquitetural da GPU.
- **Objetivo prático:** enquanto a versão padrão prioriza simplicidade e legibilidade, a versão Triton prioriza eficiência computacional e redução da latência em inferência.

APÊNDICE B – CÓDIGO DO KERNEL DE ATENÇÃO FUNDIDO

Este apêndice apresenta o código-fonte do kernel de atenção fundido desenvolvido neste trabalho, descrito no Capítulo 3. O kernel implementa o forward de atenção densa não causal com *softmax online*, e os acumuladores m_i , l_i e O_i são mantidos em FP32. Por eficiência, as exponenciais são calculadas em base 2 (`tl.exp2`), com o fator $\log_2 e$ incorporado à escala $1/\sqrt{d_k}$, transformação equivalente a $e^x = 2^{x \log_2 e}$.

```
@triton.jit
def _attention_fwd_kernel(
    q_ptr, k_ptr, v_ptr, out_ptr,
    # strides de Q, K, V e O nos eixos [B, H, N, D]
    stride_qb, stride_qh, stride_qn, stride_qd,
    stride_kb, stride_kh, stride_kn, stride_kd,
    stride_vb, stride_vh, stride_vn, stride_vd,
    stride_ob, stride_oh, stride_on, stride_od,
    num_heads: tl.constexpr,
    seq_len: tl.constexpr,
    sm_scale_log2: tl.constexpr,
    BLOCK_M: tl.constexpr,
    BLOCK_N: tl.constexpr,
    BLOCK_D: tl.constexpr,
    HEAD_DIM: tl.constexpr,
):
    pid_m = tl.program_id(0)
    pid_bh = tl.program_id(1)

    batch_idx = pid_bh // num_heads
    head_idx = pid_bh - batch_idx * num_heads

    offs_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
    offs_n = tl.arange(0, BLOCK_N)
    offs_d = tl.arange(0, BLOCK_D)

    q_base = batch_idx * stride_qb + head_idx * stride_qh
    k_base = batch_idx * stride_kb + head_idx * stride_kh
    v_base = batch_idx * stride_vb + head_idx * stride_vh
    out_base = batch_idx * stride_ob + head_idx * stride_oh
```

```
# Bloco de queries mantido em registradores durante toda a
# varredura de keys/values
q = tl.load(
    q_ptr + q_base + offs_m[:, None] * stride_qn
        + offs_d[None, :] * stride_qd,
    mask=(offs_m[:, None] < seq_len)
        & (offs_d[None, :] < HEAD_DIM),
    other=0.0,
)

# Acumuladores do softmax online (FP32)
m_i = tl.full((BLOCK_M,), -float("inf"), tl.float32)
l_i = tl.zeros((BLOCK_M,), tl.float32)
acc = tl.zeros((BLOCK_M, BLOCK_D), tl.float32)

for start_n in range(0, seq_len, BLOCK_N):
    k_offsets = start_n + offs_n
    k = tl.load(
        k_ptr + k_base + k_offsets[:, None] * stride_kn
            + offs_d[None, :] * stride_kd,
        mask=(k_offsets[:, None] < seq_len)
            & (offs_d[None, :] < HEAD_DIM),
        other=0.0,
    )
    v = tl.load(
        v_ptr + v_base + k_offsets[:, None] * stride_vn
            + offs_d[None, :] * stride_vd,
        mask=(k_offsets[:, None] < seq_len)
            & (offs_d[None, :] < HEAD_DIM),
        other=0.0,
    )

    qk = tl.dot(q, tl.trans(k)) * sm_scale_log2
    qk = tl.where(
        (offs_m[:, None] < seq_len)
        & (k_offsets[None, :] < seq_len),
        qk,
        -float("inf"),
    )
```

```

    )

    # Atualizacao incremental do maximo, da soma e da saida
    m_ij = tl.maximum(m_i, tl.max(qk, axis=1))
    p = tl.exp2(qk - m_ij[:, None])
    alpha = tl.exp2(m_i - m_ij)
    l_i = l_i * alpha + tl.sum(p, axis=1)
    acc = acc * alpha[:, None] + tl.dot(p.to(v.dtype), v)
    m_i = m_ij

acc = acc / l_i[:, None]

tl.store(
    out_ptr + out_base + offs_m[:, None] * stride_on
        + offs_d[None, :] * stride_od,
    acc.to(out_ptr.dtype.element_ty),
    mask=(offs_m[:, None] < seq_len)
        & (offs_d[None, :] < HEAD_DIM),
)

```

O wrapper em Python valida as entradas, calcula a grade de lançamento com um programa por bloco de queries e por par (lote, cabeça), e define os parâmetros de compilação:

```

def triton_flash_attention(q, k, v, *,
                           block_m: int = 16,
                           block_n: int = 64):
    """Run the custom dense attention forward kernel."""
    _validate_qkv(q, k, v)
    if not q.is_cuda:
        raise RuntimeError(
            "triton_flash_attention requires CUDA tensors")

    q, k, v = q.contiguous(), k.contiguous(), v.contiguous()
    batch, heads, seq_len, head_dim = q.shape

    block_d = triton.next_power_of_2(head_dim)
    out = torch.empty_like(q)
    grid = (triton.cdiv(seq_len, block_m), batch * heads)
    sm_scale_log2 = (1.0 / math.sqrt(head_dim)) * _LOG2_E

```

```
num_warps = 4 if head_dim <= 64 else 8

_attention_fwd_kernel[grid](
    q, k, v, out,
    q.stride(0), q.stride(1), q.stride(2), q.stride(3),
    k.stride(0), k.stride(1), k.stride(2), k.stride(3),
    v.stride(0), v.stride(1), v.stride(2), v.stride(3),
    out.stride(0), out.stride(1), out.stride(2),
    out.stride(3),
    heads, seq_len, sm_scale_log2,
    BLOCK_M=block_m, BLOCK_N=block_n,
    BLOCK_D=block_d, HEAD_DIM=head_dim,
    num_warps=num_warps, num_stages=3,
)
return out
```